

SCS 1309

Database Management Systems

Lecture 7 – Structured Query Language (SQL)

MVPT Lakshika,
Lecturer (Probationary),
Department of Information Systems Engineering, UCSC.
Contact: tlv@ucsc.cmb.ac.lk

Outline

Transaction Control Language (TCL)

- Transactions
- ACID Properties

Data Control Language (DCL)

- Authorization Identifiers And Ownership
- Privileges

1. Transaction Control Language (TCL)

Introduction

- TCL includes a set of **special commands** that deal with the **transactions** within the database.
- Basically, they are used to **manage transactions** within the database and **maintain the consistency** of the database.
- **Transaction** is **an action, or series of actions**, carried out by a **single user or application program**, which **reads or updates the contents** of the database.
- A transaction is a **logical unit of work** on the database.
 - It may be an entire program, a part of a program, or a single command (for example, the SQL command INSERT or UPDATE).
- Transactions access data using **read** and **write** operations.
- In the database context, the **execution of an application program** can be thought of as **one or more transactions** with **non-database processing** taking place in between.

Introduction

Examine the two relations:

Staff (staffNo, fName, lName, position, sex, DOB, salary, branchNo)

PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo, staffNo, branchNo)

(a) Update the salary of a particular member of staff given the staff number, **x**.

(b) Delete the member of staff with a given staff number **x**.

```
read(staffNo = x, salary)
salary = salary * 1.1
write(staffNo = x, salary)
```

(a)

```
delete(staffNo = x)
for all PropertyForRent records, pno
begin
    read(propertyNo = pno, staffNo)
    if (staffNo = x) then
        begin
            staffNo = newStaffNo
            write(propertyNo = pno, staffNo)
        end
    end
end
```

(b)

Introduction

Example transaction: (a) **Update** the salary of a particular member of staff given the staff number, **x**.

- High-level details of this transaction:

```
read(staffNo = x, salary)
salary = salary * 1.1
write(staffNo = x, salary)
```

Here, **read(x)** or **write(x)**: database read or write operation on a data item x.

- **Additional qualifiers** may be added as necessary.

For example, the notation **read(staffNo = x, salary)** indicates that we want to read the data item *salary* for the tuple with primary key value x.

In this example, we have a transaction consisting of **two database operations** (read and write) and a **non-database operation** ($\text{salary} = \text{salary} * 1.1$).

Introduction

A complicated transaction: (b) **delete** the member of staff with a given staff number **x**.

Except for **deleting** the tuple in the *Staff* relation,

- Find all the *PropertyForRent* tuples that this member of staff managed.
- Reassign them to a different member of staff, *newStaffNo*.

If all these updates are **not made**, referential integrity will be lost, and the database will be in an **inconsistent state**: *a property will be managed by a member of staff who no longer exists in the database.*

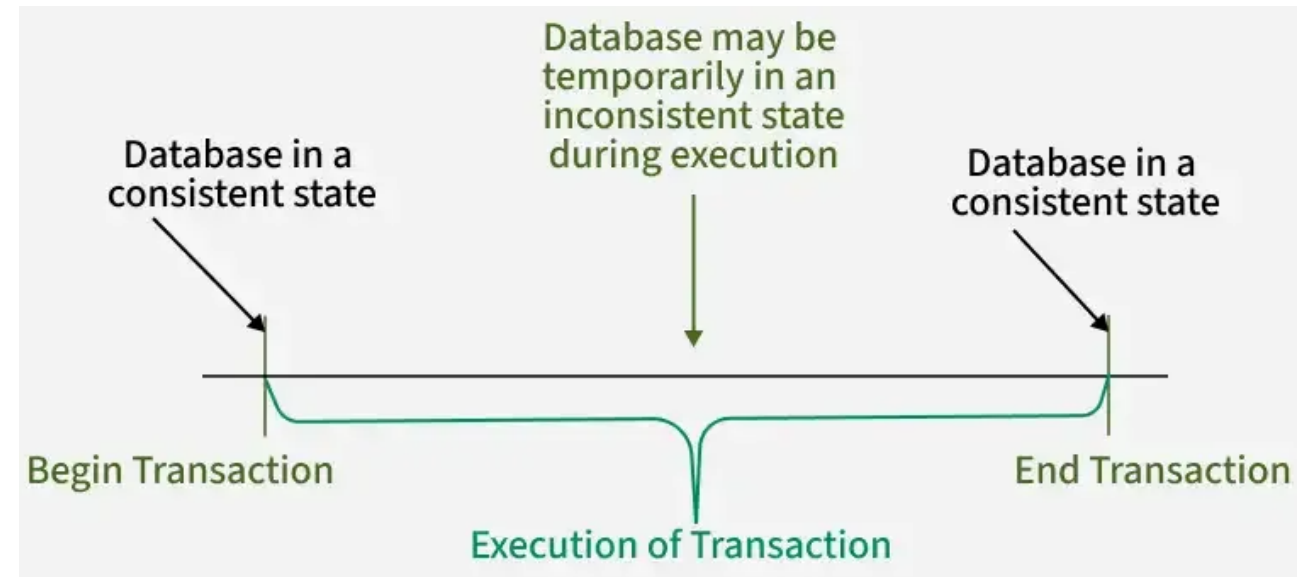
```
delete(staffNo = x)
for all PropertyForRent records, pno
begin
    read(propertyNo = pno, staffNo)
    if (staffNo = x) then
        begin
            staffNo = newStaffNo
            write(propertyNo = pno, staffNo)
        end
    end
end
```

Execution of a transaction

A **transaction** should always transform the database from one consistent state to another, although we accept that consistency may be violated while the transaction is in progress.

Example: During the transaction, there may be some moment when one tuple of *PropertyForRent* contains the new *newStaffNo* value and another still contains the old one, x.

However, at the end of the transaction, all necessary tuples should have the new *newStaffNo* value.



Execution of a transaction

- A transaction can have **one of two outcomes**.
 - **If it completes successfully**, the transaction is said to have **committed** and the database reaches a new consistent state.
 - **If the transaction does not execute successfully**, the transaction is **aborted**. If a transaction is aborted, the database must be restored to the consistent state it was in before the transaction started. Such a transaction is **rolled back** or undone.
- A committed transaction cannot be aborted.
- However, an aborted transaction that is rolled back can be restarted later and, depending on the cause of the failure, may successfully execute and commit at that time.

Delimit transactions

- The DBMS has no inherent way of knowing which updates are grouped together to form a single logical transaction.
- It must therefore provide a method to allow the user to indicate the boundaries of a transaction.
- The keywords `BEGIN TRANSACTION`, `COMMIT`, and `ROLLBACK` are available in many DMLs to delimit transactions.
- If these delimiters are not used, the entire program is usually regarded as a single transaction, with the DBMS `automatically performing a COMMIT` when the program terminates correctly and a `ROLLBACK` if it does not.

Example:

Scenario: Transferring money from one person to another.

START TRANSACTION;

-- Step 1: Check the balance first

SELECT balance **INTO** @current_balance **FROM** Accounts **WHERE** account_id = 1;

IF @current_balance >= 100 **THEN**

-- Step 2: Only update if they have enough money

UPDATE Accounts **SET** balance = balance - 100 **WHERE** account_id = 1;

UPDATE Accounts **SET** balance = balance + 100 **WHERE** account_id = 2;

COMMIT;

ELSE

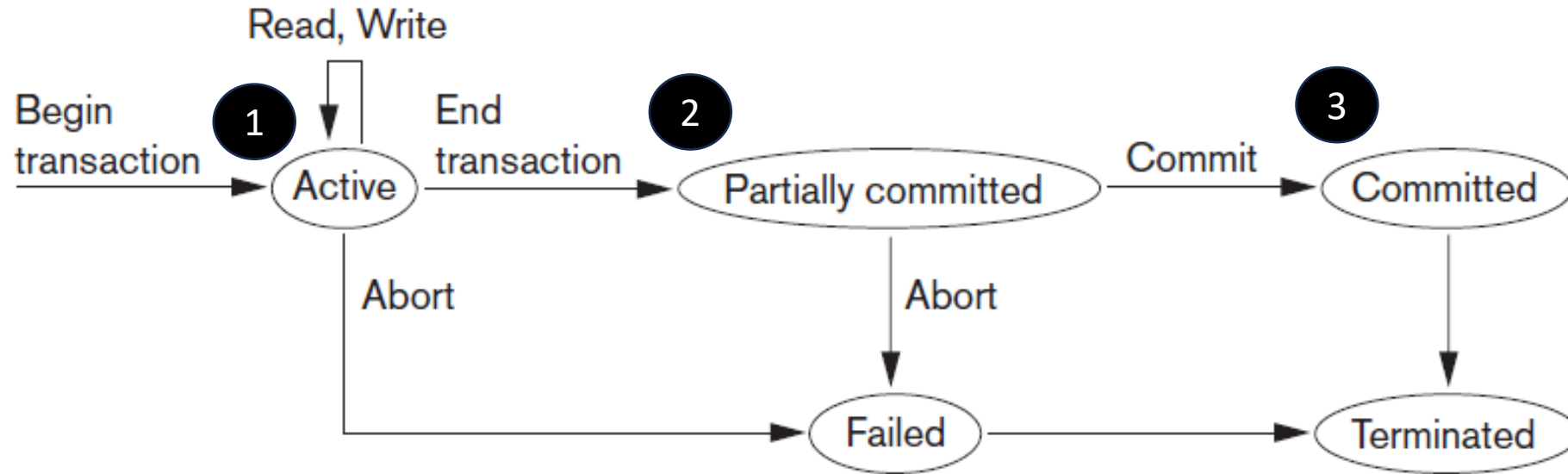
-- Step 3: Otherwise, don't even try the update

ROLLBACK;

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient funds';

END IF;

State Transition Diagram for a Transaction

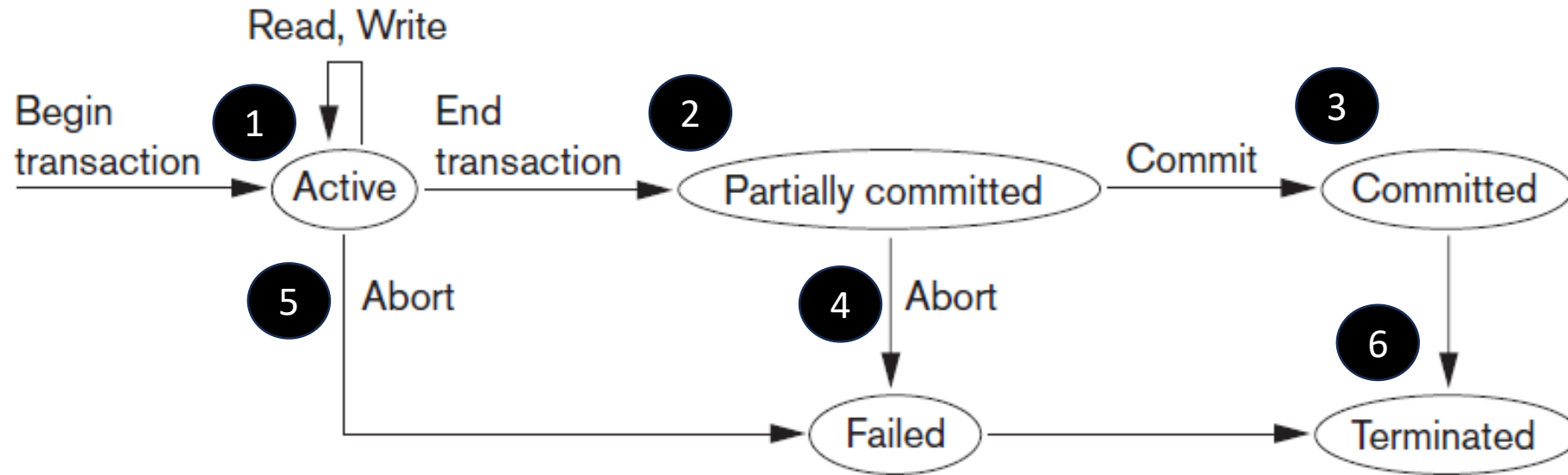


Active: It is the first state during the execution of a transaction. A transaction is active as long as its instructions (read or write operations) are performed.

Partially committed: A change has been executed in this state, but the database has not yet committed the change to disk. In this state, data is stored in the memory buffer, and the buffer is not yet written to disk.

Committed: In this state, all the transaction updates are permanently stored in the database. Therefore, it is not possible to roll back the transaction after this point.

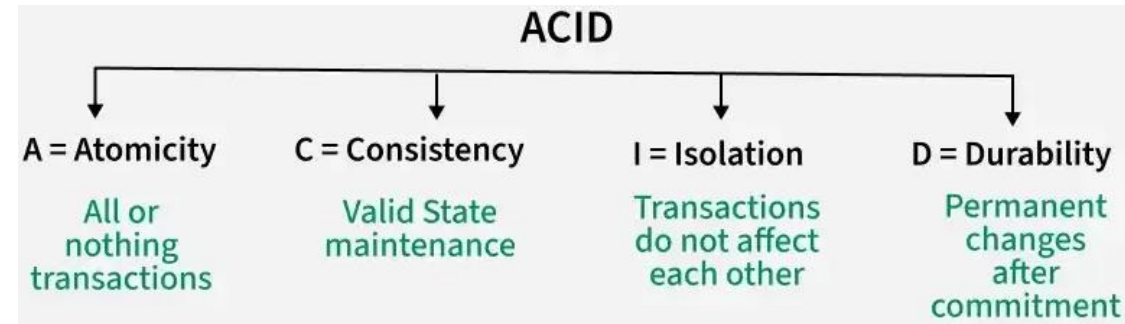
State transition diagram for a transaction



Failed: If a transaction fails or has been aborted in the active state or partially committed state, it enters into a FAILED state.

Terminated : This is the last and final transaction state after a committed or aborted state. This marks the end of the database transaction life cycle.

Properties of Transactions



- There are **properties** that all transactions should possess (the four basics are known as **ACID**).

Atomicity: A transaction is an indivisible unit that is either performed **in full or not performed at all**.

Consistency: A transaction must transform the database **from one consistent state to another consistent state**. The DBMS can ensure consistency by enforcing all the constraints specified on the database schema, such as integrity and enterprise constraints.

Isolation: Transactions **execute independently** of one another. In other words, the partial effects of incomplete transactions should not be visible to other transactions. It is the responsibility of the concurrency control subsystem to ensure isolation.

Durability: The effects of a successfully completed (committed) transaction are permanently recorded in the database and must not be lost because of a subsequent failure.

2.Data Control Language (DCL)

Introduction

- DBMS should provide a mechanism to ensure that **only authorized users can access the database**.
- DCL controls the **database maintenance and manages users' rights** in a database.
 - People who have complete rights to a database are database administrators who can manage user access.
- SQL supports only **Discretionary Access Control (DAC)** through the **GRANT** and **REVOKE** statements.

GRANT

Gives user access privileges

REVOKE

Removes privileges

- **Mandatory access control** (each database object is assigned to classification levels, including Top Secret, Secret, Confidential, Unclassified) is **not supported** in SQL.

Discretionary Access Control (DAC)

- Each user is given appropriate access rights (or *privileges*) on specific database objects.
- Typically, users obtain certain privileges when they create an object and can pass some or all of these privileges to other users at their discretion.
- Although flexible, this type of authorization mechanism can be circumvented by a devious unauthorized user tricking an authorized user into revealing sensitive data.
- The mechanism is based on the concepts of **authorization identifiers**, **ownership**, and **privileges**.

Authorization identifiers and ownership

- An **authorization identifier** is a normal SQL identifier that is used to establish the identity of a user.
- Each database user is assigned an authorization identifier by the DBA.
 - Usually, the identifier has an associated password, for security reasons.
- The authorization identifier is used to determine which database objects the user may reference and what operations may be performed on those objects.
- Each object that is created in SQL has an owner.
 - The owner is identified by the authorization identifier defined in the AUTHORIZATION clause of the schema to which the object belongs.

Privileges

- Privileges are the **actions that a user is permitted to carry out** on a given base table or view.
- The privileges defined by the ISO standard are:
 - SELECT** – the privilege to retrieve data from a table
 - INSERT** – the privilege to insert new rows into a table
 - UPDATE** – the privilege to modify rows of data in a table
 - DELETE** – the privilege to delete rows of data from a table
 - REFERENCES** – the privilege to reference columns of a named table in integrity constraints
 - USAGE** – the privilege to use domains, collations, character sets, and translations.
- The **INSERT**, **UPDATE**, and **REFERENCES** privileges can be restricted to specific columns of the table.

Granting Privileges to Other Users

- The **GRANT** statement is used to *grant privileges on database objects to specific users*.
- Normally, the GRANT statement is used by the owner of a table to give other users access to the data.
- The format of the GRANT statement is:

```
GRANT  {PrivilegeList | ALL PRIVILEGES}
ON      ObjectName
TO      {AuthorizationIdList | PUBLIC}
[WITH GRANT OPTION]
```

PrivilegeList consists of one or more of the following privileges separated by commas:

```
SELECT
DELETE
INSERT      [(columnName [, ... ])]
UPDATE      [(columnName [, ... ])]
REFERENCES  [(columnName [, ... ])]
USAGE
```

ObjectName can be the name of a base table, view, domain, character set, collation, or translation.

The **WITH GRANT OPTION** clause allows the user(s) in *AuthorizationIdList* to pass the privileges they have been given for the named object on to other users.

Example 1: GRANT all privileges

Give the user with authorization identifier **Manager** full privileges to the **Staff** table.

GRANT ALL PRIVILEGES

ON Staff

TO Manager WITH GRANT OPTION;

- The user identified as **Manager** can now retrieve, insert, update, and delete rows from the Staff table.
- Due to **WITH GRANT OPTION**, the Manager can pass these privileges on to other users.

Example 2: GRANT specific privileges

Give users **Personnel** and **Director** the privileges **SELECT** and **UPDATE** on the column *salary* of the *Staff* table.

GRANT SELECT, UPDATE (salary)

ON Staff

TO Personnel, Director;

- The keyword **WITH GRANT OPTION** is omitted here, so that users **Personnel** and **Director** cannot pass either of these privileges on to other users.

Example 3: GRANT specific privileges to PUBLIC

Give **all users** the privilege **SELECT** on the *Branch* table.

GRANT SELECT

ON Branch

TO PUBLIC;

- The use of the keyword **PUBLIC** means that all users (now and in the future) can retrieve all the data in the Branch table.
- It does not make sense to use **WITH GRANT OPTION** in this case: as every user has access to the table, there is no need to pass the privilege on to other users.

Revoking Privileges from Users

- The **REVOKE** statement is used to take away privileges that were granted with the GRANT statement.

- The format of the statement is:

```
REVOKE [GRANT OPTION FOR] {PrivilegeList | ALL PRIVILEGES}  
ON      ObjectName  
FROM    {AuthorizationIdList | PUBLIC} [RESTRICT | CASCADE]
```

- The **RESTRICT** and **CASCADE** qualifiers operate exactly as in the DROP TABLE statement.

Examples

REVOKE specific privileges from PUBLIC

Revoke the privilege **SELECT** on the Branch table from **all users**.

```
REVOKE SELECT  
ON Branch  
FROM PUBLIC;
```

REVOKE specific privileges from named user

Revoke all privileges you have given to **Director** on the *Staff* table.

```
REVOKE ALL PRIVILEGES  
ON Staff  
FROM Director;
```

This is equivalent to **REVOKE SELECT . . .**, as this was the only privilege that has been given to **Director**.

Any Questions?